

**Department of Information Science and Engineering****Online Certification Course Report**

Name of the Student/ USN	B S SHREESHA 1JS25SDS01
Course Code	MSDS202
Course Title	Deep Learning
Semester/Section	II A
Name of the Subject Faculty	Dr. Basamma Umesh Patil
Title of certification Course	[2026] Tensorflow 2: Deep Learning & Artificial Intelligence
Duration of course(hours)	26 Hours
Reference Link /Source	https://www.udemy.com/course/deep-learning-tensorflow-2/

Description about the course and key takeaways:

The course titled: [2026] Tensorflow 2: Deep Learning & Artificial Intelligence conducted by **Lazy Programmer Inc** on **Udemy Learning Platform** imparts knowledge on Tensorflow and its usage in Deep Learning and Artificial Intelligence Applications.

The course starts with installation of Tensorflow 2 on the **Google Colaboratory**. Once installed, different methods to load data into the working environment were taught.

All code files are available on GitHub: Follow the link <https://github.com/bsshreeshas/Udemy-TensorFlow2>.

Deep Learning: Introduction

Deep learning is a subset of machine learning that focuses on training artificial neural networks with multiple layers to learn complex patterns in data. With the rise of big data and computational power, frameworks like TensorFlow have enabled scalable and efficient model development.

Machine Learning

Machine Learning (ML) is a branch of artificial intelligence that enables systems to automatically learn patterns from data and make predictions or decisions without being explicitly programmed.

Instead of writing rules manually, ML models **learn a mapping function**:

- Input to Output
- Example: House features to Price prediction

Types of Machine Learning

➤ Supervised Learning

- Uses labeled data
- Examples: Regression, Classification

➤ Unsupervised Learning

- Finds patterns in unlabeled data
- Examples: Clustering, Dimensionality Reduction

➤ Reinforcement Learning

- Learns through interaction and reward signals

Neural Networks and Deep Learning

A Neural Network is a computational model inspired by the human brain, consisting of interconnected processing units called neurons arranged in layers. Each neuron computes a weighted sum of inputs, adds a bias, and applies an activation function to produce an output, enabling the network to learn patterns from data.

Deep Learning is a subfield of machine learning that uses deep (multi-layer) neural networks to automatically learn hierarchical feature representations from large datasets. It enables high-performance learning in complex tasks such as image recognition, speech processing, and natural language understanding.

Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the biological nervous system, particularly the human brain. It consists of interconnected processing units called artificial neurons, which work together to process information, learn from data, and make decisions. ANNs are widely used in machine learning and deep learning for tasks such as classification, prediction, pattern recognition, image processing, and speech recognition.

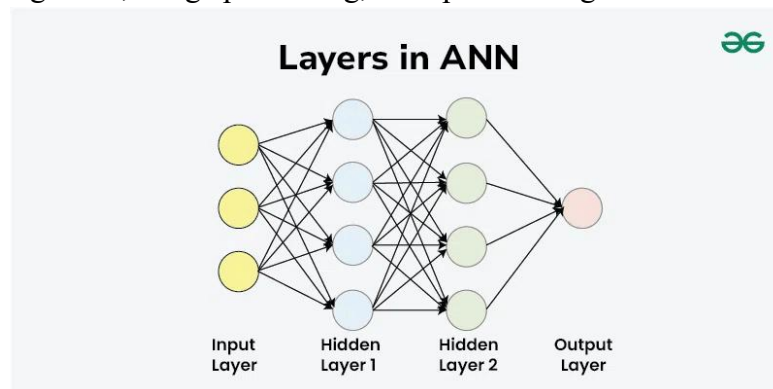


Figure 1: Diagram of ANN

Structure of ANN

An artificial neuron is a mathematical abstraction of a biological neuron and consists of:

- Inputs ($x_1, x_2, \dots, x_n$)
- Weights ($w_1, w_2, \dots, w_n$)
- Bias (b)
- Activation Function (ϕ)
- Output (y)

Mathematical Model of Artificial Neuron

$$z = \sum_{i=1}^n w_i x_i + b$$
$$y = \phi(z)$$

Activation Function and Non-Linearity

An activation function is a mathematical formula applied to the output of a neural network neuron, determining whether it should "fire" or stay inactive. It introduces non-linearity, allowing the model to learn complex, non-linear patterns in data, rather than just simple linear relationships.

Common activation functions explored include:

- **Sigmoid**: Useful for binary classification but suffers from vanishing gradients
- **ReLU (Rectified Linear Unit)**: Widely used due to computational efficiency
- **Tanh**: Centers data around zero, improving convergence in some cases

These functions introduce non-linearity, allowing the model to approximate complex functions.

Linear and Logistic Regression

Before diving into deep networks, the section revisits classical models:

- **Linear Regression** predicts continuous values using a linear relationship
- **Logistic Regression** predicts probabilities for classification tasks

Linear regression follows:

$$y = wx + b$$

Loss Functions and Optimization

To evaluate model performance, loss functions are used:

- **Mean Squared Error (MSE)** for regression
- **Binary Cross-Entropy** for classification

Optimization is performed using Gradient Descent, where model parameters are updated iteratively to minimize loss:

$$w = w - \alpha \frac{\partial L}{\partial w}$$

The learning rate α controls how quickly the model updates its parameters.

Using TensorFlow, the concepts are implemented programmatically. The workflow typically includes:

- Data preprocessing and normalization
- Model definition using Keras API
- Compilation with optimizer and loss function
- Training using `.fit()`
- Evaluation on test data

This bridges the gap between theoretical understanding and practical application.

Feedforward Neural Networks

Feedforward Artificial Neural Networks (ANNs) are one of the most fundamental architectures in deep learning. In this model, information flows in a single direction from the input layer to the output layer without any loops or feedback connections.

This section builds upon the concept of artificial neurons and extends it into multi-layer networks capable of learning complex patterns from data.

Convolutional Neural Networks

Introduction to Image Representation

Before understanding Convolutional Neural Networks, it is important to understand how images are represented mathematically.

Types of Images

There are 2 kinds of images

- a. Grayscale Images
- b. Colour (RGB) images

Grayscale images: A grayscale image is represented as a 2D matrix

$$H \times W$$

Each pixel contains an intensity value. Pixel values range from $[0, 255]$ where 0 is black and 255 is white.

Colour Images: A color image is represented as a **3D matrix**

$$H \times W \times 3$$

Where 3 is red, green, blue ranging between $[0, 255]$ and each pixel having (R, G, B)

Normalization (Important)

In deep learning, pixel values are often scaled to:

$$[0, 1]$$

by dividing by 255, which improves training stability.

Why CNN Instead of ANN?

Using Artificial Neural Networks directly on images leads to:

- Very large number of parameters
- Loss of spatial structure
- High computational cost

CNN solves this by:

- Preserving spatial relationships
- Sharing weights (filters)
- Extracting meaningful features

Convolution Operation

The main operation in CNN is convolution, where a small matrix (filter) slides over the image.

$$(I \times K) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

Where,

I = Input image

K = Filter (kernel)

Output = Feature map

Filters and Feature Maps

Filters are used to detect features such as:

- Edges
- Lines
- Textures

Each filter produces one **feature map**, and multiple filters help extract multiple features.

Output Dimension Formula

The output size after convolution depends on:

$$Output = \frac{(N - F + 2P)}{S} + 1$$

Where

N = Input Size

F = Filter Size

P = Padding

S = Stride

Pooling Layer

Pooling reduces the spatial size of the feature map.

Types of Pooling Layers

- Max Pooling
- Average Pooling

Benefits:

- Reduces computation
- Prevents overfitting

CNN Architecture

A typical CNN consists of:

1. Convolution Layer
2. Activation (ReLU)
3. Pooling Layer
4. Fully Connected Layer

Final output is passed through:

- Softmax for classification

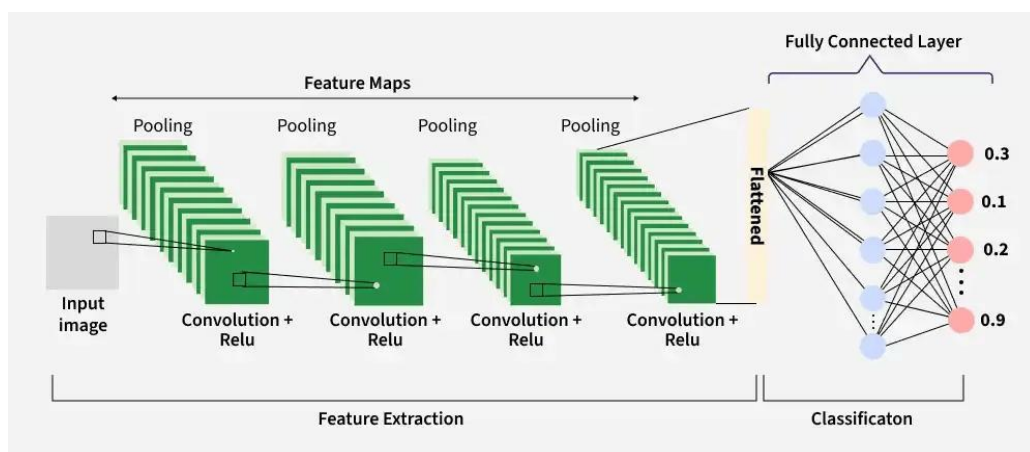


Figure 2: CNN Architecture

Implementation Using TensorFlow

Using TensorFlow:

- Conv2D for convolution
- MaxPooling2D for pooling
- Flatten for conversion

- Dense layers for classification

Datasets used:

- MNIST
- CIFAR-10

CNNs are highly effective for image processing tasks due to their ability to capture spatial features and reduce computational complexity. Understanding pixel representation, value ranges, and convolution operations is essential for building strong computer vision models.

Recurrent Neural Networks (RNN) & Long Short-Term Memory (LSTM)

Recurrent Neural Networks (RNNs) are designed to handle **sequential data**, where the order of inputs matters. Unlike feedforward networks, RNNs maintain a **memory (hidden state)** that captures information from previous time steps.

Typical applications include:

- Time series forecasting
- Natural language processing
- Speech recognition

Why Not Use ANN for Sequences?

Standard neural networks treat inputs as independent, which leads to:

- Loss of temporal relationships
- No memory of previous inputs

RNNs solve this by introducing a **loop structure**, allowing information to persist across time steps.

Mathematical Representation of RNN

At each time step t :

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Where,

x_t : Input at time t

h_t : Hidden state (memory)

W_h, W_x : Weight matrices

b : Bias

Output

$$y_t = W_y h_t + b_y$$

Unrolling Through Time

RNN can be visualized as a chain of repeated networks:

- Each step takes input + previous state
- Shares same weights across time

This is called **unrolling**, which helps in training using backpropagation through time (BPTT).

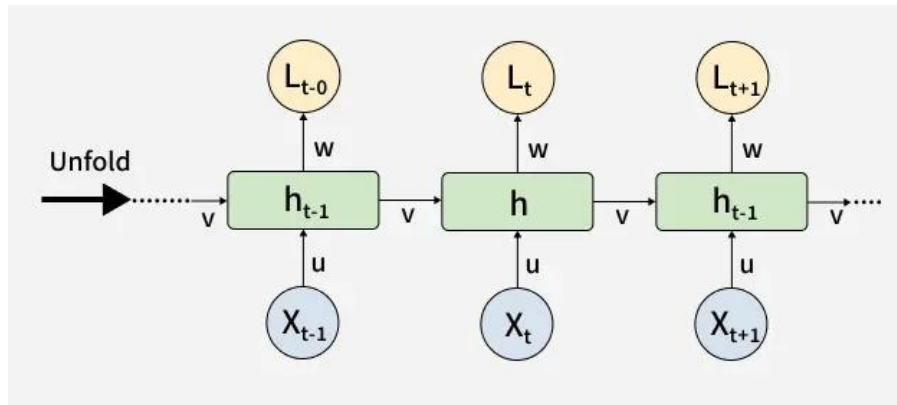


Figure 3: RNN Architecture

Problems with RNN

Vanishing Gradient Problem

- Gradients become very small
- Model fails to learn long-term dependencies

Exploding Gradient

- Gradients become too large
- Training becomes unstable

Because of this, standard RNN struggles with long sequences.

Introduction to LSTM

Long Short-Term Memory (LSTM) networks are designed to overcome RNN limitations.

They introduce a **cell state** that carries long-term information and use **gates** to control information flow.

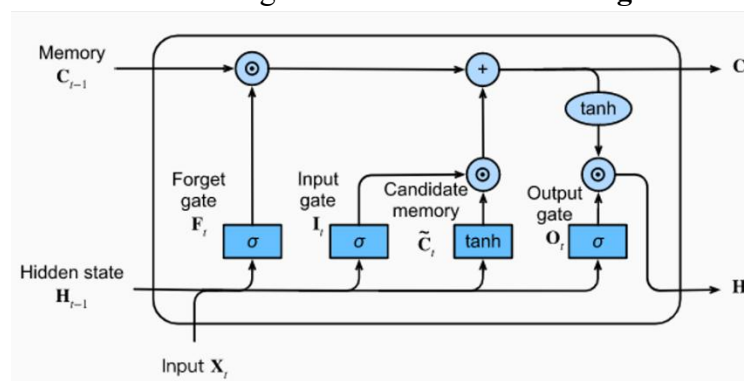


Figure 4: LSTM Architecture

LSTM Architecture

LSTM consists of:

- **Forget Gate** → Decides what to remove
- **Input Gate** → Decides what to store
- **Output Gate** → Controls output

Equation

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

Where:

C_t : Cell state

f_t : Forget gate

i_t : Input gate

$\tilde{C}_t$: Candidate values

Intuition of LSTM

- LSTM acts like a **smart memory system**
- It decides:
 - What to remember
 - What to forget
 - What to output

This allows it to capture **long-term dependencies** effectively.

Implementation Using TensorFlow

Using TensorFlow:

- SimpleRNN for basic RNN
- LSTM for advanced sequence modeling
- Embedding layer for text data

Workflow:

- Prepare sequence data
- Define model
- Train using `.fit()`
- Evaluate predictions

Natural Language Processing

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on enabling machines to understand, interpret, and generate human language. It plays a crucial role in applications such as chatbots, translation systems, sentiment analysis, and search engines.

Challenges in NLP

Unlike structured data, text data presents several challenges:

- Language ambiguity (same word, different meanings)
- Variable sentence lengths
- Context dependency
- Lack of numerical representation

To overcome these, text must be converted into numerical form before feeding into neural networks.

Text Preprocessing

Before training models, text data is cleaned and prepared.

Common Steps:

- Lowercasing text
- Removing punctuation and stopwords
- Tokenization (splitting into words)
- Padding sequences to fixed length

Text Representation (Vectorization)

Neural networks require numerical inputs, so words are converted into vectors.

Methods:

1. **One-Hot Encoding**
 - Each word → vector with one “1” and rest “0”
 - High dimensional and inefficient
2. **Word Embeddings**

- Dense vector representation
- Captures semantic meaning

Example:

- “king” and “queen” have similar vectors

Embedding Layer

An embedding layer learns word representations during training.

$$E: \mathbb{R}^V \rightarrow \mathbb{R}^d$$

Where:

- V : Vocabulary size
- d : Embedding dimension

This transforms words into meaningful dense vectors.

Sequence Modeling in NLP

Since text is sequential, models like RNN and LSTM are used.

Input sentence:

$$[w_1, w_2, w_3, \dots, w_n]$$

Each word is processed step-by-step, maintaining context across the sequence.

NLP Tasks Covered

Sentiment Analysis

- Classifies text as positive or negative
- Example: Movie reviews

Text Classification

- Categorizes text into topics
- Example: Spam vs non-spam

Next Word Prediction

- Predicts next word in a sentence

Model Architecture

Typical NLP model:

1. Embedding Layer
2. LSTM / RNN Layer
3. Dense Layer
4. Softmax Output

This architecture captures both meaning and sequence information.

Implementation Using TensorFlow

Using TensorFlow:

- Tokenizer for text processing
- `pad_sequences` for uniform length
- Embedding for vector representation
- LSTM for sequence learning

Workflow:

- Prepare text data
- Convert to sequences
- Train model
- Evaluate accuracy

Recommender Systems

Implementation Using TensorFlow

Using TensorFlow:

- Tokenizer for text processing
- pad_sequences for uniform length
- Embedding for vector representation
- LSTM for sequence learning

Workflow:

- Prepare text data
- Convert to sequences
- Train model
- Evaluate accuracy

Importance of Recommender Systems

In real-world applications, users are often overwhelmed by large amounts of data. Recommender systems help by:

- Personalizing user experience
- Increasing user engagement
- Improving business outcomes (sales, retention)

Example:

- Suggesting movies on Netflix
- Recommending products on Amazon

Types of Recommender Systems

Content-Based Filtering

- Recommends items similar to what the user liked before
- Uses item features

Collaborative Filtering

- Uses behavior of multiple users
- Finds patterns across users

Hybrid Systems

- Combines both approaches

Matrix Factorization

The interaction matrix is decomposed into two smaller matrices:

$$R \approx U \cdot V^T$$

Where:

- R : User-item matrix
- U : User feature matrix
- V : Item feature matrix

This helps in predicting missing values (recommendations).

Model Architecture

A typical deep learning recommender system includes:

1. User embedding layer
2. Item embedding layer
3. Dot product or concatenation
4. Dense layers
5. Output prediction

Implementation Using TensorFlow

Using TensorFlow:

- Embedding layers for users and items
- Custom model for interaction
- Loss function: Mean Squared Error or Binary Cross-Entropy

Workflow:

- Prepare user-item data
- Encode IDs
- Train model
- Predict recommendations

Transfer Learning

Transfer Learning is a powerful deep learning technique where a model trained on one task is reused for another related task. Instead of training a model from scratch, knowledge from a **pre-trained model** is leveraged to improve performance and reduce training time.

Motivation for Transfer Learning

Training deep neural networks from scratch requires:

- Large datasets
- High computational power
- Long training time

Transfer learning solves this by:

- Using pre-trained weights
- Reducing training time
- Improving accuracy with limited data

Pre-trained Models

Pre-trained models are trained on large datasets such as ImageNet and can be reused.

Common models include:

- VGG (Visual Geometry Group)
- ResNet (Residual Networks)
- Inception

These models have already learned general features like:

- Edges
- Shapes
- Textures

Key Approaches in Transfer Learning

Feature Extraction

- Use pre-trained model as a fixed feature extractor
- Only train final classification layers

Fine-Tuning

- Unfreeze some layers of the pre-trained model
- Retrain them along with new layers

Model Architecture

Typical transfer learning pipeline:

1. Load pre-trained model (without top layer)
2. Freeze base layers
3. Add new dense layers
4. Train on new dataset

Implementation Using TensorFlow

Using TensorFlow:

- Load models from `tf.keras.applications`
- Freeze layers using `trainable = False`
- Add custom layers using `Dense`
- Compile and train

Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a class of deep learning models used to generate new data that resembles real data. Unlike traditional models that focus on prediction, GANs are designed for **data generation**, such as creating realistic images, videos, or text. GANs consist of two neural networks that compete during training, making them highly effective for learning complex data distributions.

Core Idea of GAN

A GAN consists of two components:

- **Generator (G)**
Generates fake data from random noise
- **Discriminator (D)**
Distinguishes between real and fake data

These two networks are trained simultaneously in a **game-like (adversarial) process**.

Mathematical Objective

The objective of GAN is:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

Where:

- $D(x)$: Probability that input is real
- $G(z)$: Generated data from noise
- z : Random input

Working Process

Training GAN involves the following steps:

1. Generator creates fake data from random noise
2. Discriminator evaluates real vs fake data
3. Discriminator learns to classify correctly
4. Generator improves to fool the discriminator

This process continues until generated data becomes highly realistic.

Intuition

GAN works like a competition:

- Generator: tries to create realistic data
- Discriminator: tries to detect fake data

Over time:

- Generator becomes better at fooling
- Discriminator becomes better at detecting

Eventually, the generator produces near-real data

Model Architecture

Generator:

- Dense / Conv layers
- Upsampling
- Produces image

Discriminator:

- Convolution layers
- Downsampling
- Outputs probability (real/fake)

Implementation Using TensorFlow

Using TensorFlow:

- Define generator and discriminator models
- Use separate loss functions
- Train both models alternately

Workflow:

- Generate fake samples
- Train discriminator
- Train generator
- Repeat

Applications of GAN

- Image generation
- Image-to-image translation
- Data augmentation
- Style transfer
- Deepfake creation

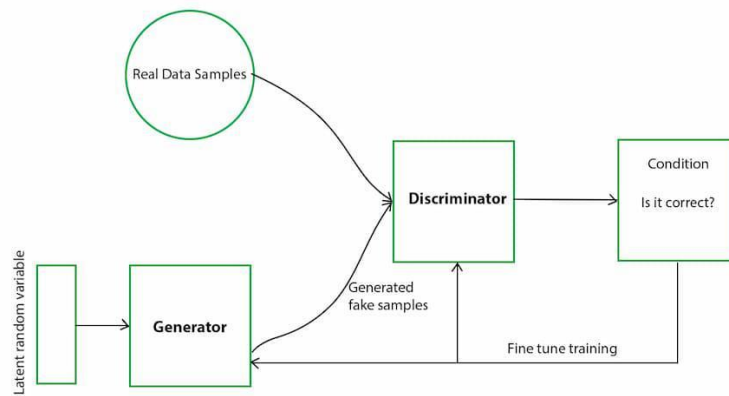


Figure 5: GAN Architecture

Challenges in GAN

- Training instability
- Mode collapse (limited diversity)
- Sensitive to hyperparameters
- Difficult to evaluate performance

Reinforcement Learning

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties. Unlike supervised learning, RL does not rely on labelled data but instead learns through trial and error.

Key Components of Reinforcement Learning

An RL system consists of:

- **Agent:** Learner or decision-maker
- **Environment:** The system the agent interacts with
- **State (S):** Current situation of the environment
- **Action (A):** Decision taken by the agent
- **Reward (R):** Feedback received after action

Learning Process

The agent follows a loop:

1. Observe current state
2. Take an action
3. Receive reward
4. Update strategy
5. Move to next state

The goal is to maximize cumulative reward over time.

Exploration vs Exploitation

A key challenge in RL:

- **Exploration:** Trying new actions
- **Exploitation:** Using known best actions

A balance between both is necessary for optimal learning.

Stock Trading Problem Setup

In this project:

- **State:** Stock prices, indicators
- **Action:** Buy, Sell, Hold
- **Reward:** Profit or loss

The agent learns when to buy or sell stocks to maximize profit.

Implementation Using TensorFlow

Using TensorFlow:

- Neural networks approximate Q-values
- Experience replay improves learning
- Target networks stabilize training

Workflow:

- Initialize agent
- Simulate trading episodes
- Update Q-values
- Track rewards

Advanced TensorFlow

Advanced TensorFlow focuses on moving beyond high-level APIs and understanding how to build more flexible and optimized deep learning models. While earlier sections used the convenient Keras interface, this section explores customization and deeper control over training and model behavior using TensorFlow.

Need for Advanced TensorFlow

High-level APIs like Keras are easy to use but have limitations:

- Limited control over training steps
- Difficulty implementing custom algorithms
- Restricted flexibility

Advanced TensorFlow allows:

- Custom training loops
- Fine-grained control over gradients
- Performance optimization

Eager Execution

TensorFlow 2 uses **Eager Execution** by default.

Features:

- Operations run immediately
- Easy debugging
- Python-like behaviour

Example:

- No need to define computational graphs beforehand

Custom Training Loop

Instead of using `.fit()`, training can be written manually.

◆ Steps:

1. Forward pass

2. Compute loss
3. Calculate gradients
4. Update weights

This gives complete control over training logic.

Gradient Computation

Gradients are computed using automatic differentiation.

Tool Used:

- GradientTape

Example concept:

- Track operations
- Compute derivatives
- Update weights

This allows efficient optimization of models.

TensorFlow Functions

TensorFlow provides `@tf.function` to convert Python functions into optimized graphs.

Benefits:

- Faster execution
- Better performance
- Graph-level optimizations

Performance Optimization

Advanced TensorFlow includes techniques to improve performance:

- GPU acceleration
- Batch processing
- Efficient data pipelines
- Parallel computation

Implementation Approach

Using TensorFlow:

- Define custom models
- Use GradientTape for training
- Apply optimizers manually
- Monitor loss and metrics

Low-Level TensorFlow

Low-Level TensorFlow focuses on understanding the core mechanics of deep learning by working directly with tensors and operations without relying on high-level abstractions like Keras. This approach provides maximum control over how models are built and trained using TensorFlow.

What are Tensors?

A tensor is the fundamental data structure in TensorFlow.

Types of Tensors:

- Scalar (0D) → Single value
- Vector (1D) → List of values
- Matrix (2D) → Grid of values

- Higher-dimensional tensors (3D, 4D, etc.)

Example:

- Image → 3D tensor
- Batch of images → 4D tensor

Tensor Operations

TensorFlow performs computations using tensor operations such as:

- Addition
- Multiplication
- Matrix multiplication
- Reshaping

These operations form the basis of neural network computations.

Computational Graph Concept

TensorFlow internally represents computations as a **graph**:

- Nodes: Operations
- Edges: Data (tensors)

This allows:

- Efficient computation
- Parallel execution
- Optimization

Manual Model Building

In low-level TensorFlow, models are built manually:

- Define variables (weights and bias)
- Perform forward computation
- Calculate loss
- Compute gradients
- Update parameters

This approach removes abstraction and exposes the inner workings of training.

Gradient Calculation

Gradients are computed using automatic differentiation.

Concept:

- Track operations
- Compute derivatives of loss
- Update weights accordingly

This ensures efficient learning.

Advantages of Low-Level TensorFlow

- Full control over computations
- Better understanding of deep learning
- Ability to implement custom algorithms
- High flexibility

Limitations

- More complex to implement
- Requires deeper mathematical understanding
- Time-consuming compared to high-level APIs

Observations

- Helps in understanding internal mechanics
- Useful for debugging complex models
- Provides flexibility for research work
- Less efficient for rapid development

Low-Level TensorFlow provides a deep understanding of how neural networks operate at their core. While it requires more effort compared to high-level APIs, it is essential for advanced users who want full control and insight into model behavior. This section completes the journey from basic machine learning concepts to advanced deep learning implementation.